



openJiuwen: Beyond Static Harnesses for Long-Horizon Coding Agents

openJiuwen Team

Huawei Technologies Co., Ltd.

Source Code: <https://github.com/openJiuwen-ai/jiuwenswarm>

Abstract

Long-horizon coding agents operate over evolving repository states while increasingly relying on heterogeneous capabilities, delegated agents, and multi-agent coordination. These trends pose two complementary challenges for the agent harness. First, developers need to compose capabilities, re-configure execution logic, and scale increasingly complex agent systems without repeatedly rebuilding orchestration. Second, complex coding tasks continuously produce new evidence—such as semantic diagnostics, execution outcomes, task progress, and changing context relevance—that should dynamically influence subsequent runtime decisions. We characterize these challenges as **Structural Composability** and **Runtime Adaptivity**. We present **openJiuwen**, an **open-source harness** designed for both developer composability and adaptive task execution. openJiuwen provides a shared execution substrate and Rail-based capability composition across single agents, delegated sub-agents, and Swarm Flow, enabling developers to construct sophisticated agent harnesses under common execution semantics. It further adapts framework-controlled runtime decisions around a fixed model policy, allowing evolving evidence to dynamically affect context, feedback, and task control toward successful completion. We systematically evaluate openJiuwen on **SWE-bench Verified** and **Terminal-Bench 2.1**, where it achieves **82.6%** and **87.19%**, respectively, exceeding the strongest selected official-leaderboard point estimates by **3.4** and **3.39 percentage points**. These results show that openJiuwen achieves strong performance on complex coding tasks while providing a composable and adaptive harness design.

Keywords: Long-Horizon Coding Agents, Agent Harness, Structural Composability, Runtime Adaptivity, Large Language Models

1. Introduction

Large language models (LLMs) are transforming automated software engineering from single-turn code generation into *long-horizon* coding agents that interact with repositories, tools, and execution environments over extended trajectories. Complex coding issues require more than locally correct actions: an agent must maintain coherent behavior while code state, diagnostics, task progress, and relevant context evolve throughout the run. As agents incorporate planning, memory, verification, context engineering, delegation, and multi-agent coordination, the *agent harness* increasingly becomes a first-class systems layer that determines how these capabilities are assembled and how execution is controlled.

Recent agent harnesses already expose increasingly sophisticated mechanisms, but they organize them through distinct architectural choices. **Pi**¹ emphasizes a minimal, provider-flexible coding runtime whose advanced behaviors are introduced via extensions. **DeepSeek Harness**² pushes further toward plugin-level composability, making core components—such as model adapters, tools, session states, and

¹<https://github.com/earendil-works/pi>

²<https://github.com/deepseek-ai/deepseek-harness>

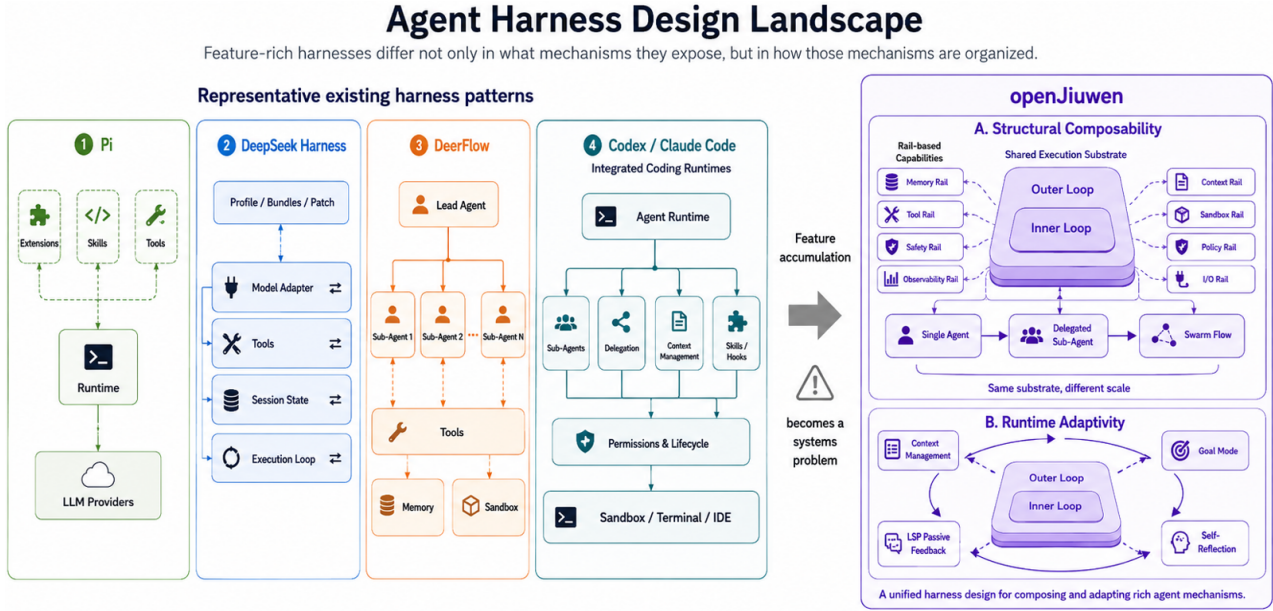


Figure 1: Architectural patterns of representative agent harnesses and the design of openJiuwen. Existing harnesses expose increasingly rich mechanisms through different architectural choices, while openJiuwen organizes harness complexity around Structural Composability and Runtime Adaptivity.

the execution loop—fully replaceable. **DeerFlow**³ focuses on long-horizon hierarchical orchestration through a lead agent and delegated sub-agents, while **Codex**⁴ and **Claude Code**⁵ integrate delegation, context management, skills/hooks, granular permissions, and lifecycle controls into mature developer-facing coding systems. Together, these systems demonstrate that the harness is no longer merely a thin tool wrapper around an LLM. As agent harnesses accumulate increasingly rich features, the harness itself becomes a new systems problem: how to organize, compose, and adapt these mechanisms coherently throughout long-horizon execution.

The first challenge is therefore **developer complexity**. As agent harnesses accumulate increasingly rich features, extensibility alone does not ensure that these capabilities remain easy to organize and combine. Plugins, hooks, sub-agents, and workflow operators may each be individually extensible, yet integrating them across different coordination patterns can introduce fragmented execution paths and bespoke orchestration logic. The key question is therefore whether capabilities and agent topologies can be composed over a common execution model rather than requiring separate execution semantics for each configuration. We refer to this property as **Structural Composability**: the ability to assemble and reconfigure capabilities and execution units over a shared execution substrate, from a single agent to delegated sub-agents and programmable multi-agent flows. Its goal is developer-facing: to let increasingly sophisticated agent systems be constructed and evolved without coupling each new configuration to a separate execution architecture.

The second challenge is **execution uncertainty**. For complex coding issues, critical evidence is revealed only while the task is being solved, including semantic diagnostics, test outcomes, intermediate results,

³<https://github.com/bytedance/deer-flow>

⁴<https://github.com/openai/codex>

⁵<https://github.com/anthropics/claude-code>

progress signals, and changes in context relevance or resource pressure. Decisions made before execution therefore become progressively stale. A harness must use newly available evidence to adapt framework-controlled decisions such as context construction, semantic feedback, task continuation, and stopping. We refer to this capability as **Runtime Adaptivity**. Its goal is task-facing: to steer the execution trajectory toward successful task completion by adapting runtime decisions around a fixed model policy as the task state evolves.

To address these challenges, we present **openJiuwen**, an **open-source harness** organized explicitly around Structural Composability and Runtime Adaptivity, as illustrated in Figure 1. For **Structural Composability**, openJiuwen provides a shared Inner Loop/Outer Loop execution substrate together with the Rail mechanism for attaching capabilities through lifecycle hooks, ordered composition, and visibility gating. The same execution semantics are reused across single agents, delegated sub-agents, and Swarm Flow, allowing developers to scale agent structure without introducing a separate execution engine at each level. For **Runtime Adaptivity**, openJiuwen changes framework-controlled runtime state rather than model parameters: Context Management adapts the information exposed to the model, Goal Mode controls task acceptance and stopping, LSP-Driven Passive Feedback injects filtered semantic diagnostics into subsequent execution, and Self-Reflection distills completed trajectories into reusable experience for later tasks. Together, these mechanisms provide a unified harness architecture for composing complex agent structures and adapting their execution as runtime evidence emerges.

Our contributions are summarized as follows:

- **Developer-facing structural composability.** We introduce a shared execution architecture that combines nested Inner Loop/Outer Loop control with Rail-based capability composition and reuses the same execution semantics across single agents, delegated sub-agents, and Swarm Flow. This design reduces the need for topology-specific execution engines and lowers the integration and orchestration burden of constructing and evolving sophisticated agent harnesses.
- **Task-oriented runtime adaptivity.** We formulate runtime adaptation as changes to framework-controlled execution state around a fixed model policy and instantiate it through Context Management, Goal Mode, LSP-Driven Passive Feedback, and Self-Reflection. These mechanisms allow evidence emerging during complex coding trajectories to influence subsequent execution through context construction, semantic feedback, continuation and stopping, and cross-task experience reuse.
- **Evaluation on complex agent tasks.** We systematically evaluate openJiuwen on **SWE-bench Verified** and **Terminal-Bench 2.1**, where it achieves **82.6%** and **87.19%**, respectively, exceeding the strongest selected official-leaderboard results by **3.4** and **3.39 percentage points**. Further analysis on SWE-bench Verified shows that openJiuwen remains competitive as task duration increases, supporting its effectiveness on long-horizon coding tasks.

2. Related Work

2.1. Coding-Agent Architectures and Scaffolds

Recent work has advanced repository-level coding agents through different scaffold designs. SWE-agent [Yang et al. \[2024\]](#) develops dedicated interfaces for repository interaction, while CodeAct and OpenHands [Wang et al. \[2024, 2025\]](#) provide executable action spaces and general-purpose development environments. Other systems strengthen specific software-engineering capabilities, including program-structure-aware localization in AutoCodeRover [Zhang et al. \[2024\]](#), repository exploration in LingmaAgent [Ma et al. \[2025\]](#), autonomous repair planning in RepairAgent [Bouzenia et al. \[2025\]](#), fixed repair pipelines in Agentless [Xia et al. \[2025a\]](#), and test-time generation and selection in Trae

Agent [Trae Research Team et al. \[2025\]](#). Beyond single-agent scaffolds, Swarm Skills [Zhang et al. \[2026\]](#) represents multi-agent roles, workflows, and coordination constraints as portable, self-evolving specifications. These approaches primarily improve particular interfaces, capabilities, or workflows; comparatively less attention has been paid to the harness as a reusable execution substrate for composing heterogeneous capabilities and scaling shared execution semantics from single agents to delegated and multi-agent execution.

2.2. Runtime Behavior for Long-Horizon Coding Agents

Long-horizon coding agents often revisit prior states or repeat work during execution. Empirical studies identify recurring action patterns and show that failed trajectories are typically longer and more variable even when relevant files have already been localized [Lindenbauer et al. \[2025\]](#); [Majgaonkar et al. \[2025\]](#). Moving from diagnosis to intervention, Ledger [Wang et al. \[2026b\]](#) maintains explicit execution state over observations, modifications, and prior actions, using it both to inform subsequent decisions and to mediate redundant commands. Collaborative Optimization [Zhang et al. \[2025a\]](#) further optimizes agent workflows through joint adaptation of workflow structures, prompts, demonstrations, and routing decisions. These works highlight the importance of runtime control beyond isolated model reasoning, but primarily focus on particular execution-state or workflow-optimization mechanisms. Long-horizon coding additionally exposes evolving semantic feedback, task progress, and context relevance, motivating broader mechanisms that adapt execution as new information emerges.

2.3. Context Management for Long-Horizon Agents

Another line of work manages long-horizon execution by controlling what information remains visible to the model. SWE-AGILE [Lian et al. \[2026\]](#) compresses older reasoning while retaining recent interactions, SWE-Pruner [Wang et al. \[2026a\]](#) selects task-relevant context, and AgentDiet [Xiao et al. \[2026\]](#) removes redundant or expired trajectory content; simple observation masking can also achieve competitive efficiency [Bouzenia and Pradel \[2025\]](#). Prompt-side adaptation provides another mechanism for changing model-visible inputs: P3 [Zhang et al. \[2025b\]](#) jointly optimizes system and user prompts and further performs query-dependent online prompting. These methods demonstrate the importance of adapting model inputs, but primarily operate on what is presented to the model. Long-horizon execution also produces evolving semantic feedback, task-progress signals, and reusable experience, motivating runtime adaptation beyond model-input adaptation alone.

Positioning of openJiuwen. openJiuwen complements these efforts by treating the harness itself as the primary systems layer. It provides a shared execution substrate for composing and scaling agent capabilities, while allowing runtime feedback, progress, and context changes to directly shape subsequent execution.

3. Method: Composable Structure and Adaptive Runtime Control

openJiuwen is organized around two complementary dimensions. *Structural composability* governs how capabilities and agent structures are composed over a shared execution substrate, from single agents to sub-agents and multi-agent systems. *Runtime adaptivity* governs how execution changes as new information emerges, including context, semantic feedback, task progress, and reusable experience. The former defines how the system is composed; the latter adapts behavior over trajectories produced by that structure.

Figure 2 provides a system-level view of both dimensions and their shared execution substrate.

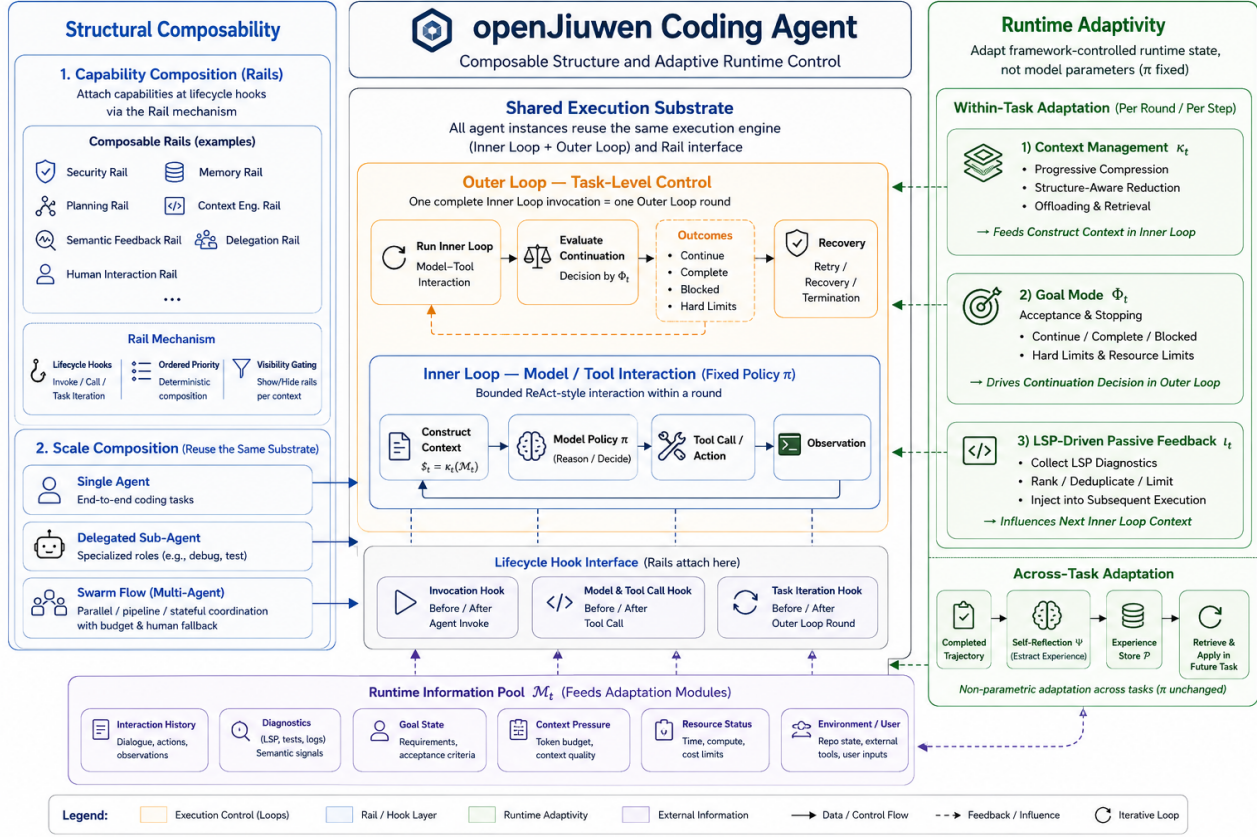


Figure 2: System overview of openJiuwen. Structural composability combines capability composition through the Rail mechanism with scale reuse from single agents and delegated sub-agents to Swarm Flow. All agent instances reuse the shared Inner Loop/Outer Loop execution core, while runtime adaptivity is realized through Context Management, Goal Mode, LSP-Driven Passive Feedback, and Self-Reflection. The illustrated Rails and runtime mechanisms are representative rather than exhaustive.

3.1. Structural Composability

Every agent instance—whether standalone, delegated as a sub-agent, or invoked within a Swarm Flow—reuses the same two primitives: the Inner Loop/Outer Loop execution engine of § 3.1.1 and the Rail-based capability interface of § 3.1.2. Scaling from one agent to many therefore changes how agents and capabilities are composed without introducing a separate agent execution engine.

3.1.1. Inner Loop and Outer Loop: Layered Execution

openJiuwen separates agent execution into two nested loops with distinct responsibilities. The Inner Loop handles model–tool interaction, while the Outer Loop handles task-level continuation and execution control.

Inner Loop. The Inner Loop follows a bounded ReAct-style interaction. At each step, the framework constructs the model-visible context, invokes policy π , executes requested tool calls, records their observations, and exposes those observations to subsequent model steps. The interaction continues until the model returns an answer, execution is interrupted, or the configured step limit is reached.

Lifecycle callbacks are triggered around model and tool boundaries, allowing Rails to observe or modify execution without embedding capability-specific logic in the loop itself.

Outer Loop. One complete Inner Loop invocation constitutes an Outer Loop round. After each round, the framework evaluates whether execution should terminate or another round should begin. The Outer Loop therefore handles task-level continuation rather than step-level reasoning.

Continuation decisions are supplied through composable evaluators, including semantic completion signals, resource limits, and user-defined conditions. Goal Mode provides a stronger task-level completion mechanism and is described in § 3.2.3. The Outer Loop also provides a boundary for execution-level control such as exceptions and bounded retry or recovery, separating infrastructure failures from additional agent reasoning.

Input arriving during an active model interaction is incorporated at a later execution boundary rather than mutating an in-flight call. The two loops thus maintain separate control granularities: the Inner Loop governs local reasoning and tool use, while the Outer Loop governs task-level continuation and recovery.

3.1.2. Rail: Ordered Capability Composition

Cross-cutting capabilities such as security, memory, planning, context engineering, semantic feedback, delegation, and human interaction require visibility into execution without becoming part of the core execution algorithm. openJiuwen therefore exposes lifecycle hooks and implements such capabilities as Rails attached to these hooks.

Let

$$\mathcal{H} = \mathcal{H}_{\text{invoke}} \cup \mathcal{H}_{\text{call}} \cup \mathcal{H}_{\text{task}} \quad (1)$$

denote the lifecycle-hook space, corresponding respectively to invocation-level, model-/tool-call-level, and task-iteration-level events.

A Rail is represented abstractly as

$$\rho = (\mathcal{H}_\rho, f_\rho, p_\rho), \quad (2)$$

where $\mathcal{H}_\rho \subseteq \mathcal{H}$ specifies the lifecycle boundaries to which the Rail attaches, f_ρ is its handler, and p_ρ is its declared priority.

At each lifecycle hook, attached Rails execute according to their declared priorities, with equal-priority cases resolved deterministically. When the merge policy permits replacement, this ordering also provides controlled overriding between capabilities. Composition order is therefore explicit metadata rather than an accidental consequence of capability-specific branches inside the execution loop.

Adding or removing a capability changes the Rail configuration while leaving the Inner Loop/Outer Loop unchanged. Model- and tool-exception hooks similarly provide uniform interception points through which Rails can participate in failure handling while the framework retains control of execution.

Capability gating. Rail composition is complemented by visibility gating. Let $g(\rho, u) \in \{0, 1\}$ indicate whether Rail ρ is visible to execution subject u . The subject-specific capability configuration is

$$\mathcal{R}(u) = \{\rho \in \mathcal{R} \mid g(\rho, u) = 1\}. \quad (3)$$

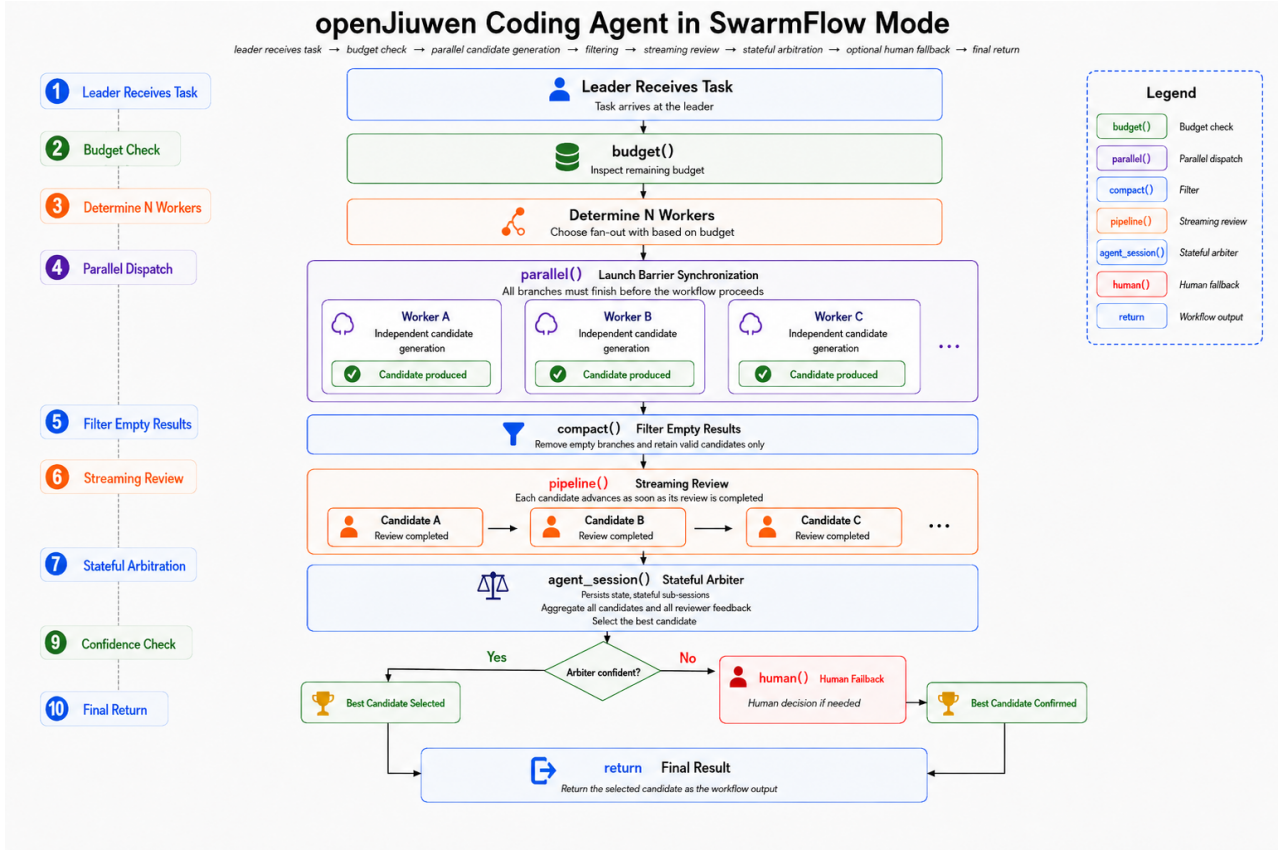


Figure 3: A Coding Agent implemented with openJiuwen Swarm Flow. The example illustrates how the provided coordination operators can be composed into a task-specific multi-agent workflow.

The same principle applies to tools. Standalone agents, delegated sub-agents, leaders, and other participating agents can therefore share the same execution substrate while exposing different capabilities. Visibility gating supports bounded recursive delegation, progressive capability disclosure, and role-specific capability isolation without changing the underlying execution engine.

3.1.3. Swarm Flow: Composable Multi-Agent Coordination

Swarm Flow extends the shared execution substrate to multi-agent systems through a set of composable coordination operators. Rather than prescribing a fixed multi-agent architecture, it allows developers to construct task-specific coordination flows by combining operators for resource control, parallel execution, result processing, stateful coordination, and fallback.

The core operators include:

- `budget()` exposes the remaining execution budget to downstream control logic;
- `parallel()` dispatches multiple agent branches concurrently and synchronizes their completion;
- `compact()` filters invalid or empty branch results before downstream processing;
- `pipeline()` streams intermediate results through subsequent processing stages as they become available;
- `agent_session()` maintains a stateful agent session across multiple stages of the flow;
- `human()` introduces optional human intervention when automated coordination is insufficient;

- `return` terminates the flow and exposes the final result.

Figure 3 shows one concrete Coding Agent built with these operators. The leader queries `budget()` and uses the remaining budget to determine the number of workers. `parallel()` launches these workers to generate independent candidates, while `compact()` removes empty results and `pipeline()` streams valid candidates through review. A stateful arbiter implemented with `agent_session()` aggregates candidate results and reviewer feedback. If its confidence is insufficient, the flow can invoke `human()` before producing the final `return`.

This Coding Agent is one realization of Swarm Flow. Developers can rearrange, omit, and combine these operators to construct their own multi-agent coordination strategies.

3.2. Runtime Adaptivity

Structural composability determines how agents and capabilities are assembled, whereas runtime adaptivity determines how their effective behavior changes as execution unfolds. openJiuwen considers adaptation at two time scales. Within a task, the runtime adjusts context construction, semantic feedback, and acceptance/stopping decisions according to newly available execution evidence. Across tasks, completed trajectories can be distilled into reusable experience for later executions.

Both forms of adaptation operate around a fixed model policy π . openJiuwen changes framework-controlled runtime state rather than model parameters.

3.2.1. Constrained Online Runtime Adaptation

We use constrained online optimization as a conceptual lens for single-task runtime adaptivity rather than as a numerical optimization algorithm. At runtime control epoch t , let $\mathcal{M}_t$ denote the information currently available to the framework, including interaction history, diagnostics, goal state, context pressure, and resource status. Context construction produces

$$c_t = \kappa_t(\mathcal{M}_t), \quad (4)$$

and the fixed model policy acts according to

$$a_t \sim \pi(\cdot \mid c_t), \quad \pi \text{ fixed.} \quad (5)$$

We summarize the framework-controlled runtime configuration as

$$\Theta_t = (\kappa_t, \Phi_t, \iota_t), \quad (6)$$

where κ_t controls context construction, Φ_t controls semantic acceptance and stopping, and ι_t controls diagnostic-feedback injection. The subscript t denotes the effective state-dependent behavior of each mechanism, rather than a newly learned mechanism at every epoch.

The conceptual value of an execution trajectory is

$$J(\Theta_{0:T}) = \mathbb{E} \left[\sum_{t=1}^T (r_t - \lambda \text{cost}_t) \right], \quad \lambda \geq 0, \quad (7)$$

where r_t and $\text{cost}_t \geq 0$ denote conceptual task-utility and runtime-cost contributions at epoch t , respectively. The stopping time T is determined by the acceptance and resource conditions formalized in § 3.2.3.

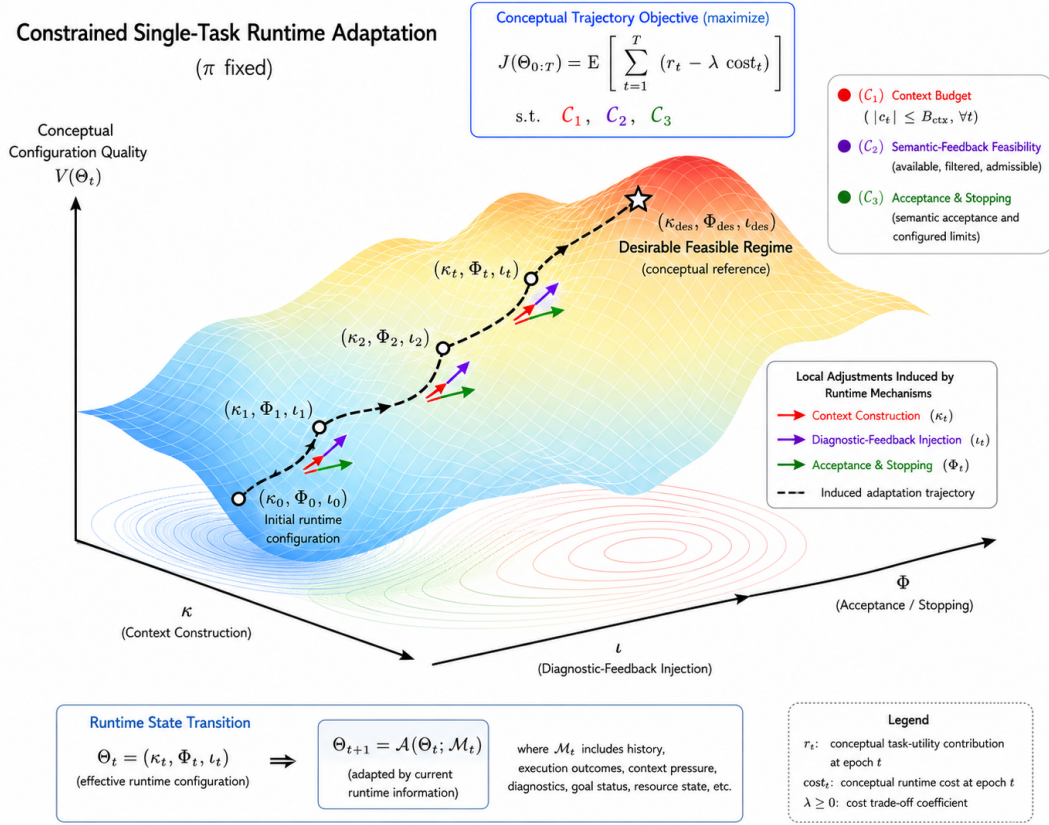


Figure 4: Conceptual view of constrained single-task runtime adaptation under a fixed model policy π . An effective runtime configuration $\Theta_t = (\kappa_t, \Phi_t, l_t)$ captures context construction, acceptance/stopping, and diagnostic-feedback injection. The surface is a schematic projection used to visualize relative configuration quality, while the trajectory-level objective $J(\Theta_{0:T})$ evaluates the execution trajectory as a whole. Runtime evidence induces state-dependent adaptation subject to context-budget, semantic-feedback, and stopping constraints.

As new evidence becomes available, the effective runtime configuration changes according to

$$\Theta_{t+1} = \mathcal{A}(\Theta_t; \mathcal{M}_t), \quad (8)$$

where $\mathcal{A}$ abstracts the state-dependent adjustments induced by the concrete runtime mechanisms described below.

These adjustments operate within three runtime feasibility constraints:

$$C_1 : \quad |c_t| \leq B_{\text{ctx}}, \quad \forall t, \quad (9)$$

which enforces the context budget; C_2 , which requires diagnostic feedback to be available, filtered, and admissible before injection; and C_3 , which requires execution to obey semantic acceptance and configured stopping limits.

Figure 4 illustrates this view. Context pressure, semantic diagnostics, execution outcomes, goal assessments, and resource state change the information available in $\mathcal{M}_t$ and can thereby induce a

transition from Θ_t to Θ_{t+1} . The objective in Equation (7) is conceptual: the concrete mechanisms implement structured online adjustments intended to steer execution toward higher-utility feasible configurations rather than directly solving a generic optimization program.

3.2.2. Context Management: Adaptive Context Construction

Context Management realizes the context-construction component κ . Rather than treating model context as a static concatenation of the system prompt and complete interaction history, openJiuwen processes available information according to current context pressure, task state, and content structure.

Registered context processors realize κ_t by selectively transforming, compressing, or offloading information in $\mathcal{M}_t$ before the final model context is assembled according to Equation (4). The resulting context remains subject to the budget constraint $\mathcal{C}_1$.

Progressive compression. Context is reduced progressively rather than maximally compressed at once. For a context unit m ,

$$m^{(0)} \rightarrow m^{(1)} \rightarrow \dots \rightarrow m^{(L)}, \quad |m^{(0)}| > |m^{(1)}| > \dots > |m^{(L)}|,$$

where later representations retain less detail. Concrete mechanisms include dialogue summarization, incremental compression of large interaction histories, and full-session compaction. Recent information is preferentially retained at higher fidelity.

Structure-aware reduction and dead-loop collapse. Structured content such as code diffs, logs, and tables can first be reduced by deterministic structure-aware processors before model-based summarization is used. Repeated unproductive reasoning or tool-call patterns can likewise be collapsed to prevent redundant history from consuming the context budget.

Offloading and retrieval. Large artifacts need not remain fully resident in active context. Their full content can be externalized while the live context retains a compact summary or handle. Context therefore follows a progressive information hierarchy:

$$\text{full content} \rightarrow \text{structured/semantic summary} \rightarrow \text{compact handle} \rightarrow \text{on-demand retrieval}.$$

For compressed dialogue, retrieval can proceed coarse-to-fine, first locating relevant rounds and then recovering finer-grained source content.

Context Management can additionally coordinate with the inference layer through session affinity and KV-cache reuse. This allows semantic context processing and inference-state management to share session-level lifecycle information, including during multi-agent execution.

3.2.3. Goal Mode: Acceptance-Constrained Stopping

Goal Mode realizes the acceptance and stopping component Φ . It maintains an explicit task goal across multiple execution attempts and determines whether task-level execution should continue.

Given the framework information available at epoch t , the semantic completion evaluator returns

$$\Phi_t(\mathcal{M}_t) \in \{\text{continue, complete, blocked}\}. \quad (10)$$

Semantic completion is separated from configured hard limits such as maximum attempts, time, or resource usage. Let $g_{\text{cap}}(\mathcal{M}_t) \in \{0, 1\}$ indicate whether such a limit has been reached. The task therefore stops at

$$T = \inf\{t: \Phi_t(\mathcal{M}_t) \neq \text{continue} \vee g_{\text{cap}}(\mathcal{M}_t) = 1\}. \quad (11)$$

This separation distinguishes successful completion or semantic blockage from resource exhaustion.

openJiuwen supports self-assessment, independent assessment, and hybrid assessment strategies. Self-assessment uses the acting agent’s completion report, independent assessment evaluates the trajectory through a separate evaluation path, and hybrid assessment combines both signals. These strategies share the same Goal Mode interface rather than introducing different execution engines.

Goal Mode operates within the current task. It is therefore distinct from Self-Reflection, which acts after a trajectory has completed and affects later tasks.

3.2.4. LSP-Driven Passive Feedback: Closed-Loop Semantic Correction

LSP-Driven Passive Feedback realizes the diagnostic-feedback component ι . Language Server Protocol infrastructure provides mechanically derived semantic evidence after relevant code changes, allowing newly exposed code issues to influence subsequent model decisions.

Let $\Delta(s_t)$ denote the diagnostics produced for code state s_t . Diagnostics are ranked, deduplicated, and bounded before exposure:

$$\delta_t = \text{RankDedupLimit}(\Delta(s_t); K_{\text{file}}, K_{\text{sem}}), \quad (12)$$

where K_{file} limits diagnostics contributed by one file and K_{sem} limits the total diagnostic volume.

The admissible feedback is then made available to subsequent execution:

$$\mathcal{M}_{t+1}^+ = \mathcal{M}_{t+1} \cup \iota_t(\delta_t), \quad (13)$$

where $\mathcal{M}_{t+1}^+$ denotes the feedback-augmented information pool. This realizes constraint $\mathcal{C}_2$: semantic feedback can affect later decisions only after it is mechanically produced and filtered by the runtime.

The same language-server infrastructure also supports active queries such as definition lookup, reference lookup, and call-hierarchy inspection. The difference is control: passive diagnostics are surfaced automatically after relevant code changes, whereas active semantic information is explicitly requested by the agent.

The scope of this mechanism is limited to properties exposed by the underlying analysis infrastructure, such as type errors, symbol relationships, and static diagnostics. It does not by itself establish architectural quality, maintainability, business-logic correctness, or end-to-end task success.

3.2.5. Self-Reflection: Cross-Task Non-Parametric Adaptation

The mechanisms above adapt execution within one task. Self-Reflection operates across tasks: after a trajectory has completed, it extracts reusable experience and makes selected experience available to future executions.

For completed task τ_n with trajectory h^{τ_n} , let

$$X_n = \Psi(h^{\tau_n}) \quad (14)$$

denote the experience extracted by the configured reflection procedure. The persistent experience store is updated through

$$\mathcal{E}_{n+1} = \mathcal{U}(\mathcal{E}_n, X_n), \quad (15)$$

where $\mathcal{U}$ may validate, deduplicate, merge, or revise extracted experience before persistence.

For a later task τ_{n+1} , let q_{n+1} denote the retrieval query. Relevant experience is retrieved as

$$R_{n+1} = \text{Retrieve}(\mathcal{E}_{n+1}, q_{n+1}), \quad (16)$$

and becomes part of the information available to the new task:

$$\mathcal{M}_0^+ = \mathcal{M}_0 \cup R_{n+1}. \quad (17)$$

Retrieval does not bypass Context Management: available experience may still be compressed, summarized, deferred, or omitted when constructing the actual model context.

Self-Reflection therefore changes the information available to future tasks without updating the model policy π . It constitutes cross-task non-parametric adaptation, complementary to the within-task runtime trajectory illustrated in Figure 4.

4. Experiments

4.1. Experimental Setup

Benchmarks. We evaluate openJiuwen on two complementary benchmarks. **SWE-bench Verified**⁶ Jimenez et al. [2024] contains 500 human-validated software-engineering tasks derived from real-world GitHub issues, requiring agents to modify existing repositories and produce patches that pass the corresponding test suites. **Terminal-Bench 2.1**⁷ Merrill et al. [2026] contains 89 diverse tasks executed in containerized terminal environments, covering broader tool-driven and long-horizon agent execution.

Models and configurations. For **SWE-bench Verified**, we evaluate openJiuwen with **Claude Opus 4.5** using the **high** reasoning-effort setting. For the primary **Terminal-Bench 2.1** result, we use **GPT-5.6 Sol**. We additionally evaluate openJiuwen with **Fable 5**, enabling a model-matched comparison with Claude Code and Terminus 2 using the same backbone model. Within each openJiuwen configuration, the prompt, available tools, and model configuration remain fixed. SWE-bench patches are graded using the official test suites, while Terminal-Bench 2.1 tasks are evaluated using their task-specific verifiers.

Metrics. For SWE-bench Verified, we report **Pass@1**, the percentage of instances successfully resolved. For Terminal-Bench 2.1, we report **Accuracy** according to the benchmark’s task-specific verifiers. For comparison with state-of-the-art systems, we use results reported on the corresponding official leaderboards.

⁶<https://www.swebench.com/>

⁷<https://www.tbench.ai/leaderboard/terminal-bench/2.1>

Table 1: Comparison with leading systems on **Terminal-Bench 2.1**. Prior results are taken from the official leaderboard.

Rank	Agent / Harness	Agent Org.	Model	Model Org.	Effort	Date	Accuracy (%)
10	Claude Code	Anthropic	Sonnet 5	Anthropic	high	2026-07-09	74.6 ± 1.6
9	Codex	OpenAI	GPT-5.6 Luna	OpenAI	max	2026-07-11	75.7 ± 1.3
6	Codex	OpenAI	GPT-5.6 Terra	OpenAI	max	2026-07-11	78.4 ± 1.3
5	Claude Code	Anthropic	Opus 4.8	Anthropic	high	2026-07-09	78.9 ± 1.3
3	Terminus 2	Terminal-Bench	Fable 5	Anthropic	high	2026-06-05	80.4 ± 1.2
2	Codex	OpenAI	GPT-5.5	OpenAI	xhigh	2026-05-01	83.1 ± 1.1
1	Claude Code	Anthropic	Fable 5	Anthropic	xhigh	2026-06-07	83.8 ± 1.2
–	openJiuwen	Ours	Fable 5	Anthropic	high	2026-08-21	84.04 ± 1.12
–	openJiuwen	Ours	GPT-5.6 Sol	OpenAI	high	2026-08-21	87.19 ± 1.20

Table 2: Comparison with leading systems on **SWE-bench Verified**. Bash-only mini-SWE-agent entries are excluded.

Rank	Agent / Harness	Org.	Model	Date	Resolved (%)
16	Lingxi-v1.5 Yang et al. [2026]	Lingxi	Claude 4 Sonnet	2025-07-20	74.60
8	ACoder	ACoder	Multiple	2025-08-19	76.40
6	EPAM AI/Run Developer Agent	EPAM	Claude 4 Sonnet	2025-08-04	76.80
4	live-SWE-agent Xia et al. [2025b]	UIUC	Gemini 3 Pro Preview	2025-11-20	77.40
3	TRAE Trae Research Team et al. [2025]	TRAE	Doubao-Seed-Code	2025-09-28	78.80
2	Sonar Foundation Agent	Sonar	Claude 4.5 Opus	2025-12-05	79.20
1	live-SWE-agent Xia et al. [2025b]	UIUC	Claude 4.5 Opus	2025-12-15	79.20
–	openJiuwen	Ours	Claude 4.5 Opus	2026-08-21	82.60

4.2. Results on Terminal-Bench 2.1

Results. As shown in Table 1, openJiuwen with GPT-5.6 Sol achieves **87.19%** accuracy on Terminal-Bench 2.1, exceeding the strongest selected result on the official leaderboard, Claude Code with Fable 5 at 83.8%, by **3.39 percentage points**. In the model-matched Fable 5 comparison, openJiuwen also achieves **84.04%**, slightly exceeding Claude Code at 83.8% by **0.24 percentage points** and outperforming Terminus 2 at 80.4%. The Fable 5 comparison reduces model capability as a confounding factor, although differences in prompts, tools, and agent implementations remain. Together, these results show that openJiuwen is competitive on broader long-horizon execution in heterogeneous terminal environments.

Category-level analysis. To examine where the overall result arises, Table 3 reports the complete breakdown across all 16 Terminal-Bench 2.1 task categories for both openJiuwen configurations, together with Claude Code and Terminus 2. Figure 5 further highlights six representative categories in which openJiuwen with Fable 5 is particularly competitive. The Fable 5 columns provide a model-matched view that reduces differences caused by the backbone model, while the GPT-5.6 Sol column shows the category-level behavior of the primary configuration.

Table 3: Category-level task success scores on **Terminal-Bench 2.1**. The full benchmark contains 89 tasks. Bold indicates the best score in each row (ties included).

Task Type	# Tasks	openJiuwen Fable 5	openJiuwen GPT-5.6 Sol	Claude Code Fable 5	Terminus 2 Fable 5
data-processing	4	1.00000	1.00000	1.00000	0.95000
data-querying	1	1.00000	1.00000	1.00000	1.00000
data-science	8	0.72500	0.95000	0.87500	0.87500
debugging	5	1.00000	0.96000	0.92000	1.00000
file-operations	5	0.76000	0.72000	0.56000	0.52000
games	1	1.00000	1.00000	0.80000	1.00000
machine-learning	3	0.86667	0.86667	1.00000	0.86667
mathematics	4	0.80000	0.95000	0.90000	0.70000
model-training	4	0.80000	0.70000	0.75000	0.70000
optimization	1	1.00000	0.80000	1.00000	1.00000
personal-assistant	1	1.00000	1.00000	1.00000	1.00000
scientific-computing	8	0.85000	0.77500	0.67500	0.72500
security	8	0.90000	0.72500	0.82500	0.80000
software-engineering	26	0.80769	0.90769	0.88462	0.80769
system-administration	9	0.88889	0.95556	0.77778	0.84444
video-processing	1	0.20000	0.40000	0.80000	0.20000

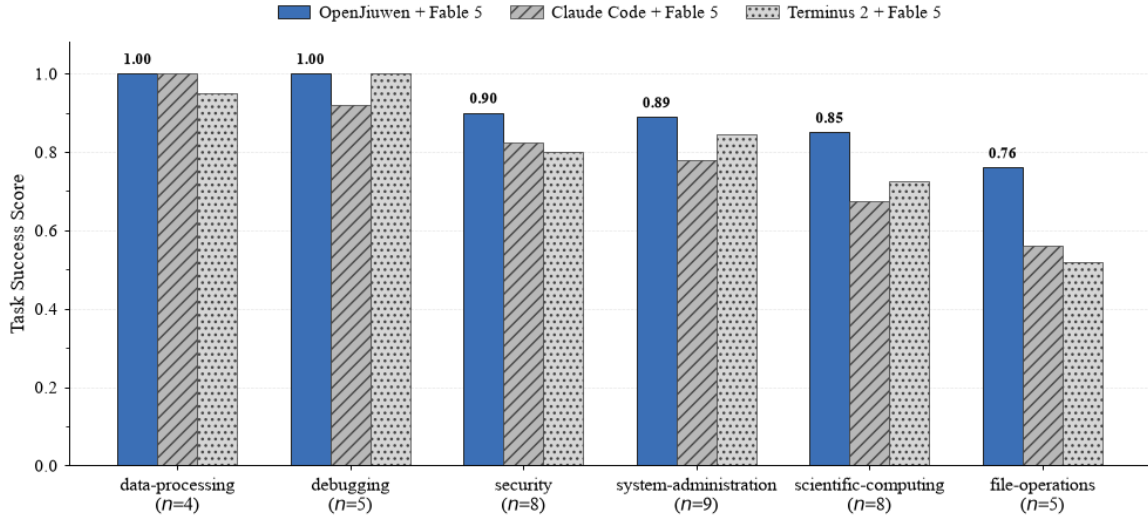


Figure 5: Model-matched comparison on representative Terminal-Bench 2.1 categories using Fable 5. openJiuwen is particularly strong on tool-intensive categories such as file operations and system administration.

Table 4: SWE-bench Verified success rates grouped by estimated fix duration. Bold indicates the best score in each row (ties included).

Fix Duration	# Tasks	Opus 4.5 (high) mini-swe-agent	Opus 4.5 (med.) mini-swe-agent	Opus 4.5 (med.) live-SWE-agent	Opus 4.5 (med.) Sonar Foundation	Opus 4.5 (high) openJiuwen
< 15 min fix	194	0.89175	0.84536	0.88660	0.88144	0.91753
15 min–1 hour	261	0.74713	0.72414	0.76628	0.77778	0.81226
1–4 hours	42	0.35714	0.42857	0.54762	0.50000	0.52381
> 4 hours	3	0.33333	0.33333	0.33333	0.33333	0.33333

The category breakdown is especially favorable on **file operations** and **system administration**. With Table 5, openJiuwen reaches 0.76 on file operations, compared with 0.56 for Claude Code and 0.52 for Terminus 2; on system administration, it reaches 0.889, compared with 0.778 and 0.844, respectively. The GPT-5.6 Sol configuration further increases the system-administration score to 0.956 and reaches 0.908 on software engineering and 0.950 on data science. These tasks are strongly tool-dependent, requiring agents to repeatedly inspect and modify files, interact with the terminal, and verify environment state. We attribute part of openJiuwen’s strength in these categories to its broad set of out-of-the-box operational tools and the unified execution interface through which these tools are exposed. This reduces the need for task-specific tool construction and allows the agent to devote more of its execution trajectory to task reasoning, action, and verification. Because the benchmark does not isolate tool availability from other harness mechanisms, this explanation should be interpreted as a plausible systems-level factor rather than a controlled causal attribution.

4.3. Results on SWE-bench Verified

Results. Table 2 compares openJiuwen with leading non-Bash-only systems on SWE-bench Verified. openJiuwen resolves **82.6%** of the 500 instances, outperforming the strongest selected leaderboard results at 79.2% by **3.4 percentage points**. Notably, the strongest selected prior systems also use Claude 4.5 Opus, suggesting that the performance difference cannot be attributed solely to the use of a stronger model. The result indicates that openJiuwen provides an effective harness for repository-level, long-horizon software-engineering tasks.

Performance by task duration. To examine robustness as the required execution horizon grows, Table 4 groups SWE-bench Verified tasks into four disjoint buckets by estimated fix duration. We focus on Claude Opus 4.5 configurations so that the comparison more directly reflects differences in the agent harness and reasoning-effort setting.

The duration breakdown shows that openJiuwen remains competitive as the execution horizon grows. It achieves the best result on both the < 15 minute bucket (91.75%) and the substantially larger 15-minute–1-hour bucket (81.23%). On the 42 tasks in the 1–4 hour bucket, openJiuwen reaches 52.38%, substantially above mini-swe-agent with the same Opus 4.5 high setting (35.71%) and also above mini-swe-agent with medium reasoning effort (42.86%); it is close to the strongest result from live-SWE-agent at 54.76%. The > 4 hour bucket contains only three tasks, so we do not draw a strong conclusion from that row alone.

An interesting pattern appears within mini-swe-agent: Opus 4.5 with *high* reasoning effort underperforms its *medium* setting on the 1–4 hour tasks (35.71% vs. 42.86%). A plausible explanation is a context-budget effect. Higher reasoning effort can spend more tokens on deliberation, and over long

trajectories this can increase pressure on the finite context window, leaving less effective capacity for repository state, tool feedback, and execution history. In contrast, openJiuwen uses Opus 4.5 with high reasoning effort yet reaches 52.38% on the same duration bucket. This result is consistent with the role of openJiuwen’s context-management mechanism, which selectively retains and exposes task-relevant information as execution proceeds, mitigating context pressure while preserving the benefit of deeper reasoning. Since the benchmark does not directly isolate reasoning-token usage or context overflow, we present this as a systems-level interpretation rather than a controlled causal claim.

Overall comparison. Across both benchmarks, openJiuwen exceeds the strongest selected leaderboard result: by **3.39 percentage points** on Terminal-Bench 2.1 and by **3.4 percentage points** on SWE-bench Verified. The two benchmarks stress substantially different execution settings—repository-grounded issue resolution and general terminal-based task execution—yet openJiuwen achieves strong performance in both. This consistency supports the design goal of openJiuwen as a reusable harness for coordinating complex agent execution across heterogeneous long-horizon workloads. Because leaderboard systems may differ in models, prompts, tools, and agent implementations, these results should be interpreted as system-level comparisons rather than controlled estimates of the contribution of the harness alone.

5. Limitations

While openJiuwen provides a general harness for structurally composable and runtime-adaptive coding agents, several directions remain for further development. First, the current design treats runtime Self-Reflection and offline evolution as complementary stages; tighter integration between them could enable more systematic reuse of execution experience. Second, Goal Mode currently focuses on task-level objectives, and extending it to hierarchical goals may further improve control over complex, multi-stage tasks. Third, future work can further automate the selection and coordination of Context Management strategies under different runtime conditions. Finally, our current evaluation focuses on SWE-bench Verified and Terminal-Bench 2.1. Broader controlled studies across additional benchmarks, models, and configurations, together with more detailed ablations, would further characterize the generality and contributions of individual harness mechanisms.

6. Conclusion

We presented **openJiuwen**, an **open-source coding-agent harness** that addresses two complementary challenges: **Structural Composability** and **Runtime Adaptivity**. For developers, openJiuwen provides a shared **Inner Loop/Outer Loop** execution substrate, **Rail**-based capability composition, and reusable execution semantics across single agents, delegated sub-agents, and multi-agent coordination, reducing the engineering effort required to construct and extend sophisticated agent harnesses. For complex coding tasks, openJiuwen adapts framework-controlled runtime decisions through **Goal Mode**, **LSP-Driven Passive Feedback**, **Context Management**, and **Self-Reflection**, allowing evolving execution evidence to continuously guide subsequent behavior. Evaluations on **SWE-bench Verified** and **Terminal-Bench 2.1** demonstrate strong performance across repository-level software engineering and broader terminal-based tasks. Together, these results show that a composable harness architecture can support both efficient agent development and effective execution on complex coding tasks.

References

- Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025. doi: 10.1109/ASE63991.2025.00234.
- Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. Repairagent: An autonomous, llm-based agent for program repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200. IEEE, 2025. doi: 10.1109/ICSE55347.2025.00157.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *ICLR*, 2024.
- Shuquan Lian, Juncheng Liu, Yazhe Chen, Yuhong Chen, and Hui Li. SWE-AGILE: A software agent framework for efficiently managing dynamic reasoning context. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17536–17550, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.868.
- Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025. doi: 10.48550/arXiv.2508.21433.
- Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. Alibaba lingmaagent: Improving automated issue resolution via comprehensive repository exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pages 238–249. ACM, 2025. doi: 10.1145/3696630.3728549.
- Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. Understanding code agent behaviour: An empirical study of success and failure trajectories. *arXiv preprint arXiv:2511.00197*, 2025. doi: 10.48550/arXiv.2511.00197.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. doi: 10.48550/arXiv.2601.11868.
- Trae Research Team, Pengfei Gao, Zhao Tian, Xiangxin Meng, Xincheng Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025. doi: 10.48550/arXiv.2507.23370.

- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *ICLR*, 2025.
- Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. SWE-Pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*, 2026a. doi: 10.48550/arXiv.2601.16746.
- Zehao Wang, Yisen Xu, Chenglin Li, Chao Peng, Bram Adams, Ahmed E. Hassan, and Tse-Hsun Chen. Turning interaction history into execution state: A runtime layer for long-horizon coding agents. *arXiv preprint arXiv:2608.00808*, 2026b. doi: 10.48550/arXiv.2608.00808.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2025a.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint arXiv:2511.13646*, 2025b. doi: 10.48550/arXiv.2511.13646.
- Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. Reducing cost of llm agents with trajectory reduction. *Proceedings of the ACM on Software Engineering*, 3(FSE):1241–1263, 2026. doi: 10.1145/3797084.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.
- Xu Yang, Jiayuan Zhou, Michael Pacheco, Wenhan Zhu, Pengfei He, Shaowei Wang, Kui Liu, and Ruiqi Pan. Lingxi: Repository-level issue resolution framework enhanced by procedural knowledge guided scaling. In *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2026.
- Xinyu Zhang, Ran Dou, Enrui Hu, Minjun Zhao, Yangkai Ding, and Zhicheng Dou. Collaborative optimization approach for workflow agents in user behavior modeling. In *Companion Proceedings of the ACM Web Conference 2025*, pages 2988–2992, 2025a.
- Xinyu Zhang, Yuanquan Hu, Fangchao Liu, and Zhicheng Dou. P3: Prompts promote prompting. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11948–11965, 2025b.
- Xinyu Zhang, Zhicheng Dou, Deyang Li, Jianjun Tao, Shuo Cheng, Ruifeng Shi, Fangchao Liu, Enrui Hu, Yangkai Ding, Hongbo Wang, Qi Ye, Xuefeng Jin, and Zhangchun Zhao. Swarm skills: A portable, self-evolving multi-agent system specification for coordination engineering. *arXiv preprint arXiv:2605.10052*, 2026.
- Yuntong Zhang, Haifeng Ruan, Fan Wu, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1118–1129, 2024.

A. Author List

Core Contributors. Tao Yu, Xinyu Zhang, Qianqian Chen, Xiaoneng Xiang, Chia Kwangyang, Xingchen Huang, Ran Chen, Yangkai Ding, Zheng Wang, Yeo Boon Hong, Bingzheng Gan, Enrui Hu, Shuo Cheng, Deyang Li, Ruifeng Shi, Hongbo Wang, Qi Ye, Xuefeng Jin, Zhangchun Zhao